

## TRAVAUX PRATIQUES II

# Croquer la tablette à pleines dents

L'objectif de ce TP est d'implémenter le calcul des attracteurs sur le jeu de Chomp  $p \times q$ .  
On programme en OCaml.

## A Jeu de Chomp

### A.1 Règles du jeu

Le jeu de Chomp<sup>1</sup> est un jeu où deux joueurs se servent alternativement dans une même tablette de chocolat. La tablette est un rectangle  $p \times q$  de carrés individuels. Le carré de l'angle en haut à gauche est empoisonné.

Pour jouer un coup, un joueur choisit un carré de la tablette. Il va le manger ainsi que tous ceux situés en bas à droite de celui-ci<sup>2</sup> (cf Figure II.1).

Le joueur qui mange le carré d'angle en haut à gauche a perdu.

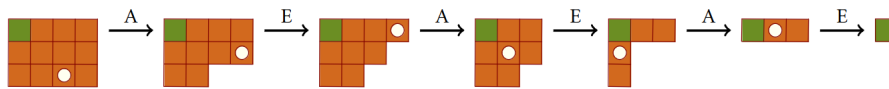


FIGURE II.1 – Une partie de Chomp entre J1 (Adam) et J2 (Eve). Le carré choisi est marqué par une pastille blanche.

À partir d'ici, sauf mention contraire, on considère une tablette  $t$  à initialement  $p$  lignes et  $q$  colonnes. On repère un carré en donnant ses deux coordonnées (*ligne, colonne*) (origine (0, 0) en haut à gauche, comme d'habitude).

### A.2 Modélisation

Une première idée naïve pour représenter la tablette serait une matrice rectangulaire. Il y a en fait plus simple : on peut remarquer qu'une ligne de la tablette n'a jamais de « trou ». Toute ligne  $i$  est composée d'abord de  $\ell_i$  carreaux intacts, puis de  $q - \ell_i$  carreaux dévorés.

0. (Bonus) Prouver cette remarque sur la structure des tablettes.

Aussi, on représentera une tablette par le tableau de ses  $\ell_i$ . Pour une configuration, on doit aussi stocker le joueur la contrôlant. Enfin, on va également stocker  $q$ , qui sera utile entre autres pour l'affichage. Cela donne le type ci-dessous :

OCaml : chomp.ml

```
1 type controle = J1 | J2
2 type config = {joueur : controle; tablette : int array; q : int}
3 (** Pour représenter une tablette, on stocke la longueur de chaque ligne
4     et la longueur initiale [q] *)
```

Une fonction `print_tablette : config -> unit` est fournie et permet d'afficher la tablette d'une configuration<sup>3</sup>.

1. Écrire une fonction `next : controle -> controle` qui à un joueur associe l'autre joueur.

À partir d'ici, on note la tablette  $[\ell_0; \dots; \ell_{p-1}]$ . Au début de la partie, pour tout  $i$ , on a  $\ell_i = q$ .

2. Représenter :

- Pour  $p = 3$  et  $q = 4$ , la tablette  $[[4; 0; 0]]$ .
- Pour  $p = 3$  et  $q = 4$ , la tablette  $[[1; 1; 1]]$ .

1. « to chomp » est un verbe anglophone désignant l'action de mordre.

2. Autrement dit, il mange le rectangle dont le carré choisi est l'angle haut gauche.

3. Elle utilise des emojis de Unicode12.0. Si l'affichage ne marche pas/mal, c'est probablement lié à votre police d'emojis.

- c. Pour  $p = 5$  et  $q = 5$ , la tablette  $[[5; 5; 1; 0; 0]]$ .

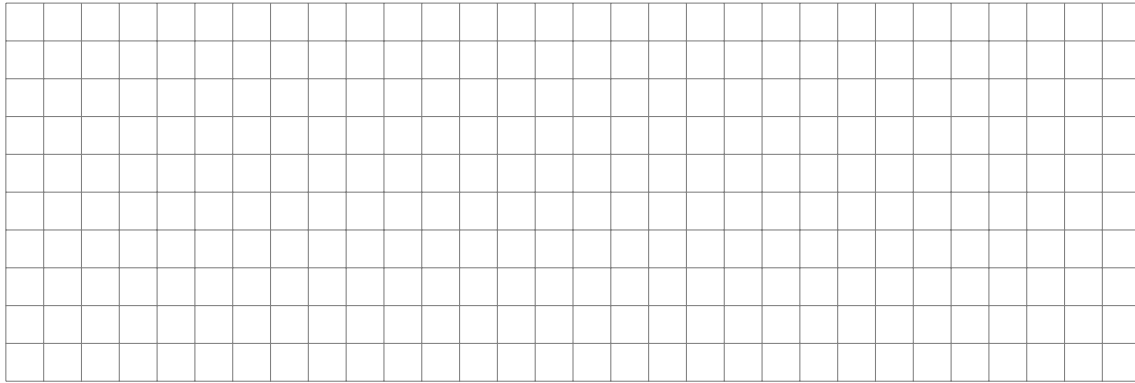


FIGURE II.2 – Du papier quadrillé pour dessiner des tablettes;

3. Comment accéder à  $p$  avec notre représentation OCaml ?
4. Écrire une fonction `config_initiale : int -> int -> config` qui renvoie la configuration initiale du Chomp  $p \times q$ .
5. Écrire une fonction `terminaux : int -> int -> joueur -> config list`. `terminaux p q j` doit être la liste des configuration(s) terminale(s) gagnante(s) pour le joueur  $j$  du Chomp  $p \times q$ .

### A.3 Premières analyses

On appelle  $croque(x, y)$  la fonction qui transforme une tablette en croquant le carré  $(x, y)$  de  $t$ , ainsi que tous ceux qui sont à la fois à sa droite et en dessous de lui.

6. On note  $croque(x, y)(t) = [[\ell'_0; \dots; \ell'_{p-1}]]$ . Exprimer les  $\ell'_i$  en fonction des  $\ell_i$ .
7. a. On admet que  $[[\ell_0; \dots; \ell_{p-1}]]$  représente bel et bien une tablette valide (obtenable durant une partie) si et seulement si pour tout  $i \in \llbracket 0; p-1 \rrbracket$ , on a  $\ell_i \in \llbracket 0; q \rrbracket$  et  $(i+1 \leq p-1 \implies \ell_i \geq \ell_{i+1})$ .  
Écrire une fonction `tablette_valide : config -> bool` qui teste si la tablette d'une configuration est valide.  
b. (Bonus) Prouver cette propriété.
8. Écrire une fonction `chomp : int -> int -> config -> config`. `chomp x y` doit implémenter la fonction  $croque(x, y)$ .  
Vous ferez bien attention à renvoyer une nouvelle configuration et à ne pas modifier celle donnée en argument.
9. (Bonus) Un peu plus d'étude théorique pour vous entraîner.
  - a. (Bonus) On note  $u(p, q)$  le nombre de tablettes  $p \times q$  valides. Trouver une formule récursive vérifiée par  $u(p, q)$ .
  - b. (Bonus) Écrire une fonction calculant efficacement  $u(p, q)$ .
  - c. (Bonus) En déduire la taille du graphe des états du Chomp  $p \times q$ .
10. (Bonus) Toujours plus.
  - a. (Bonus, difficile) À l'aide du théorème de Zermelo (TD exercice 2) et d'un vol de stratégie, déterminer quel joueur a une stratégie gagnante.  
On pourra traiter à part le cas de la tablette  $1 \times 1$ .
  - b. (Bonus, difficile) En expliciter une.

## B Calcul des attracteurs

Pour le calcul des attracteurs, on calculera explicitement le graphe des états. Les graphes seront représentés par liste d'adjacence : un tableau (associatif) qui à un état associe la liste des états voisins. Cela donne le type :

```
1 type graphe = (config, config list) Hashtbl.t
2 (** Un graphe est représenté par le tableau associatif de ses listes d'adjacences *)
```

## B.1 Tables de hachage en OCaml

Le module de tables de hachage en OCaml se nomme `Hashtbl`. On pourra utiliser toutes ses fonctions, dont notamment :

- `Hashtbl.create` . `Hashtbl.create n` crée un tableau associatif à  $n$  cellules. Il s'agrandira automatiquement si besoin est : inutile de calculer la longueur optimale. Il ne se rétrécit par contre pas automatiquement.
- `Hashtbl.find` . `Hashtbl.find t a` cherche la valeur associée à la clef  $a$  dans  $t$  (en pseudo-code, il cherche  $t[a]$ ). S'il n'y a pas de telle association, il lève `Not_found`.
- `Hashtbl.mem` . `Hashtbl.mem t a` cherche si la clef  $a$  est associée à une valeur dans  $t$  (en pseudo-code, il  $t[a]$  existe). Il renvoie `true` si c'est le cas, `false` sinon.
- `Hashtbl.replace t a b` associe  $b$  à la clef  $a$  dans  $t$ . Elle écrase l'éventuelle association précédente de  $a$ , d'où son nom<sup>4</sup>.
- `Hashtbl.iter` . `Hashtbl.mem f t` itère la fonction  $f$  sur les couples (clef, valeur) de  $t$ . Ainsi,  $f$  prend deux arguments : une clef et une valeur.

## B.2 Le calcul

11. Rappeler le pseudo-code du calcul des attracteurs.

Deux fonctions très utiles au calcul des attracteurs sont déjà codées :

- `construit_graphe : int -> int -> graphe` . `construit_graphe p q` est le graphe des états du Chomp  $p \times q$ .
- `transpose_graphe : graphe -> graphe` . `transpose_graphe g` est le graphe transposé de  $g$ .

Le code source de ces fonctions est fourni, mais ne passez pas de temps à les lire : les recoder sera une question bonus pour les plus rapides.

12. Implémentez :

- a. `decremente` telle que `decremente nb_voisins c` diminue de 1 l'entier associé à la clef  $c$  dans la table de hachage `nb_voisins`.
- b. `calcul_attracteurs` telle que `calcul_attracteurs p q j` renvoie un tableau associatif `attr` telle que pour toute configuration  $c$ ,  $c$  est associé à `true` dans `attr` si et seulement si  $c$  est attractrice pour  $j$ .

*Indication : pour parcourir une table de hachage, on ne peut pas utiliser de simple boucle `for`, il faut utiliser `Hashtbl.iter`. Définissez une fonction qui fait le corps de la boucle et donnez lui en argument. Vous pouvez même la définir séparément de `calcul_attracteurs`.*

## C Pour aller plus loin

13. Recoder les fonctions de construction du graphe et de transposition d'un graphe. Ne lisez pas la solution, bien sûr...
14. Coder une fonction `rang` qui au lieu de juste calculer si un sommet est attracteur ou non calcule son rang.
15. Coder une fonction `start` qui calcule une bonne stratégie (« bonne » étant laissé à votre interprétation).
16. Rendre le graphe de vos fonctions implicite : trouver une façon de ne pas stocker le graphe des configurations en entier, en en calculant à la volée uniquement ce qui est nécessaire quand cela l'est.

4. Sa variante `Hashtbl.add` n'écrase pas, et donc fait garder en mémoire les associations précédentes. Cela peut-être utile, mais pas dans ce TP.